Moirae: A Multimodal Agent Collaborative Framework for Dynamic Android Malware Detection

Anonymous Author(s)

Abstract

Android applications increasingly serve as gateways to Web services, making malware detection an important aspect of mobile Web endpoint security. However, the continuous evolution of Android applications and malware introduces concept drift, reducing the reliability of detectors that rely on implementation-specific code features or isolated runtime signals. In contrast, runtime behavioral context that integrates user-visible content, interaction behaviors, and low-level execution operations can provide more interpretable evidence for malware detection. However, such heterogeneous evidence differs substantially in semantic granularity and observation scope, making unified analysis challenging. Therefore, we propose Moirae, a multimodal multi-agent framework for dynamic Android malware detection. Moirae first collects visual content, UI interaction events, and API calls through dynamic execution, and organizes these heterogeneous observations using state- and event-based indices. Three specialized ReAct agents then independently investigate different evidence views and extract potential behavioral risk signals. Finally, a centralized Judge integrates their reports and assesses maliciousness within the runtime behavioral context. Experimental results show that Moirae achieves an F1 score of 92.15% without target-domain fine-tuning and improves accuracy by up to 18.40 percentage points over the best-performing trained baseline. Temporal evaluation further shows that Moirae maintains relatively stable detection performance as Android malware evolves. [1]

CCS Concepts

• **Security and privacy → Malware and its mitigation**; **Mobile platform security**.

Keywords

Android malware detection, large language models, concept drift, multi-agent systems, dynamic analysis

ACM Reference Format:

Anonymous Author(s). 2026. Moirae: A Multimodal Agent Collaborative Framework for Dynamic Android Malware Detection. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (ACM Web Conference '2027)*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

[1]The code of Moirae is released at https://anonymous.4open.science/r/Moirae-0145/.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACM Web Conference '2027, Dublin, Ireland

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Android applications increasingly provide access to Web services, making their security an important aspect of mobile Web endpoint security [5, 9]. Existing Android malware detection approaches typically extract features from permissions, API calls, and program structures, and employ supervised learning to establish statistical associations between these features and malware labels [1, 19, 31]. However, as Android applications and system interfaces evolve, malware also continuously changes its implementation techniques, behavioral triggers, and evasion strategies [6, 11]. These changes may shift feature distributions and their relationships with malware labels, leading to concept drift that can substantially degrade detection performance on applications collected in later periods or from different sources [30].

To address this challenge, prior studies have explored behavioral abstraction, semantic representations, and distribution-shift modeling to reduce detector sensitivity to specific features [25, 32, 37]. Other approaches adapt to newly emerging samples through pseudo-labeling, active learning, and continual model updates [8, 33]. Meanwhile, dynamic analysis can capture application interactions and system operations during actual execution [12, 16, 29]. More recently, the semantic understanding and reasoning capabilities of large language models (LLMs) have also been leveraged to interpret application code and program-analysis evidence [14, 26, 34, 43]. These directions provide new opportunities to move beyond individual feature patterns. Nevertheless, existing detection decisions still largely depend on specific APIs, code patterns, or localized risk signals, whose effectiveness may deteriorate as malware behavior evolves.

These limitations motivate a shift from isolated risk signals to behavioral context. The security implications of an application operation often depend on its relationship with user-visible functionality and interaction processes, rather than on the operation alone [26, 38]. For example, accessing device identifiers or requesting sensitive permissions may be legitimate, but can become suspicious when inconsistent with the application's visible functionality or user interactions [13, 15]. Moreover, although applications may implement similar behaviors through different APIs or code structures, the relationships among user-visible functionality, interactions, and underlying operations can still provide meaningful behavioral cues [35, 36]. Such contextual relationships offer a promising basis for malware detection beyond implementation-specific patterns.

However, leveraging behavioral context for malware detection presents two key challenges. First, organizing heterogeneous runtime evidence for contextual behavioral analysis is difficult. Screenshots capture user-visible content, UI states, and interaction events describe interaction processes, and Android Runtime (ART) method traces record low-level operations [20, 21]. These observations differ in semantic granularity, observation scope, and scale, complicating

the interpretation of low-level operations in relation to user-visible interactions. Analyzing evidence views in isolation or combining them without preserving their contextual relationships may obscure behavior-level risk signals [24, 42]. Second, integrating complementary evidence under partial observability remains challenging. Dynamic exploration cannot guarantee complete behavioral coverage [3, 39, 42], and missing observations should not be treated as evidence of benign behavior. Moreover, different evidence views may reveal distinct risks or provide seemingly inconsistent risk signals. Therefore, malware detection requires jointly interpreting evidence-grounded findings across views while preserving their provenance and accounting for missing observations.

To address these challenges, we propose `Moirae`, a multi-agent framework for Android malware detection through contextual reasoning over runtime evidence. `Moirae` collects screenshots, UI states, interaction events, and ART method traces during dynamic execution, and organizes them using state- and event-based indices for traceable, on-demand evidence retrieval. Three specialized ReAct agents independently investigate visual, interaction, and API evidence, producing structured findings grounded in observed behaviors. A centralized Judge then integrates these findings, considering supporting evidence, potential discrepancies, and missing observations, to produce an APK-level decision with an evidence-based rationale. By combining structured evidence organization with cross-view adjudication, `Moirae` aims to improve malware detection generalization under distribution shifts.

Our contributions are summarized as follows:

- We propose `Moirae`, a multi-agent framework for dynamic Android malware detection. To address distribution shifts and generalization challenges, `Moirae` interprets potentially malicious behaviors through complementary runtime evidence from user-visible content, interaction states, and low-level execution, reducing reliance on shallow statistical features.
- We design state- and event-indexed dynamic evidence retrieval tools, three independent modality-specialized ReAct investigation agents, and a centralized Judge Agent that aggregates structured findings, forming an analysis pipeline from heterogeneous observations to behavior-level risk interpretation and APK-level malware detection.
- Extensive experiments show that `Moirae` achieves an F1 score of 92.15% on out-of-distribution test data. Compared with the strongest baseline trained on large amounts of labeled data, `Moirae` improves accuracy by up to 18.40 percentage points under a zero-shot setting, demonstrating the effectiveness of its multi-agent architecture.

2 Related Work

We review prior Android malware detection research from three perspectives: feature-based and drift-adaptive detection, dynamic and hybrid analysis, and LLM-based malware analysis, as summarized in Figure 1.

Feature-Based and Drift-Adaptive Detection. Android malware detection has long relied on features such as permissions, API calls, and program structures [2, 10, 17, 27, 28]. However, the evolution of application implementations and attack techniques can alter both

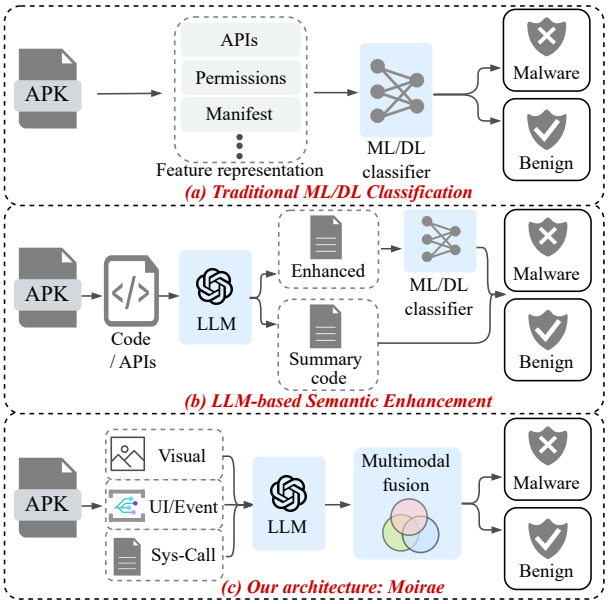


Figure 1: Comparison of different scheme architectures.

the distributions of these features and their discriminative value. To mitigate detector degradation over time, prior studies have mainly explored two directions: improving behavioral representations and adapting detection models to distribution shifts.

MaMaDroid [25] abstracts API calls to package- or family-level representations and models call transitions using Markov chains, reducing sensitivity to changes in individual APIs. SDAC [32] learns semantic distances from API-call contexts and adapts to evolving Android interfaces through clustering and feature extension for newly introduced APIs, without requiring retraining on newly labeled samples. Another line of work focuses on detecting and adapting to distribution shifts. CADE [37] uses contrastive representations to identify and explain drifted samples that deviate from previously observed classes. DroidEvolver [33] performs online updates using evolving features and pseudo-labels, thereby reducing the dependence on manual annotation. CL-Malware [8] combines contrastive learning with active learning to select newly observed samples for annotation and subsequent model updates. These studies show that drift detection, unsupervised or weakly supervised adaptation, and limited human feedback can improve long-term detection performance. However, pseudo-label-based updates may propagate incorrect predictions, while active learning requires continuous annotation and model maintenance.

Dynamic and Hybrid Analysis. Dynamic and hybrid analysis further broaden the range of available evidence. DTDroid [29] analyzes dynamic behaviors and network activities to detect Android malware. HGDetector [12] combines static function-call relationships with runtime network behaviors to obtain richer detection representations. These studies demonstrate that integrating heterogeneous evidence can compensate for the limitations of individual feature types. However, once runtime observations are transformed into vector or graph representations, the concrete relationships between

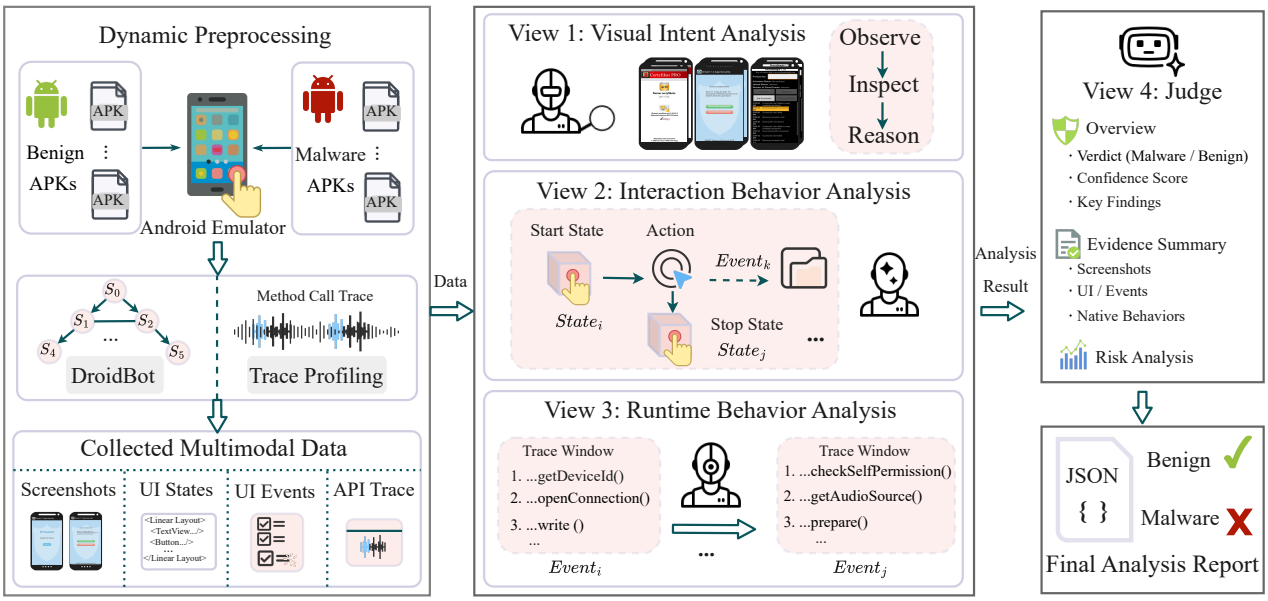


Figure 2: Overview of the Moirae architecture.

the intent of user interactions and underlying sensitive operations often remain insufficiently explained.

LLM-Based Malware Analysis. LLMs provide new opportunities for semantic analysis of program behavior. Under the semantic-enhancement paradigm, methods such as AppPoet [43] and SRDC [44] employ LLMs primarily as semantic feature enhancers, while the final detection decision still depends on downstream learning-based models that may require continual retraining. In contrast, recent studies have explored direct reasoning with LLMs. LAMD [26] extracts security-critical code contexts, performs hierarchical reasoning over program structures, and incorporates factual-consistency mechanisms to reduce unsupported conclusions. TraceRAG [41] retrieves method-level code summaries and performs multi-round analysis to locate potentially malicious implementations and generate human-readable explanations. MARD [40] adopts a multi-agent ReAct paradigm in which agents invoke static program-analysis tools on demand to support malware decisions. Nevertheless, code-level reasoning primarily analyzes implementations and potential execution paths present in static programs, making it difficult to directly capture user interactions and system operations that occur during actual execution.

3 Overview of the Moirae Architecture

To address distribution shifts and generalization challenges caused by the evolution of Android malware, we design Moirae, an Android malware detection framework based on reasoning over behavior-contextualized dynamic evidence. Moirae leverages LLMs to reason about user-visible content, interaction changes, and observed system operations during application execution, thereby producing risk assessments grounded in runtime context. For an application under analysis $\mathcal{A}$, the system first obtains a set of runtime observations $O(\mathcal{A}) = \{\mathcal{V}, \mathcal{S}, \mathcal{E}, \mathcal{T}\}$, where $\mathcal{V}$, $\mathcal{S}$, $\mathcal{E}$, and $\mathcal{T}$ denote screenshots, UI states, interaction events, and ART method traces, respectively, as illustrated in Figure 2.

Heterogeneous Dynamic Evidence Organization. The system collects multi-level runtime observations through dynamic exploration. It links screenshots, state transitions, and interaction actions using UI state identifiers and event indices, and constructs candidate indices to ART call summaries based on event time windows. The system also preserves provenance information for the original observations and explicitly records trace availability, enabling subsequent analysis to retrieve specific interfaces, events, and API records on demand.

Specialized Behavioral Evidence Investigation. To accommodate differences in semantic granularity and analysis scale across evidence views, Moirae employs three specialized ReAct agents (Vision, UI/Event, and API) with distinct tool permissions. They respectively investigate visual deception signals, UI states and interaction processes, and captured system calls. The three agents analyze the same APK in parallel but operate within independent contexts. Through paginated retrieval and iterative investigation, each agent produces a structured report containing concrete observations, provenance information, risk interpretations, and an overall risk assessment. This division of responsibilities preserves view-specific interpretations and avoids prematurely collapsing heterogeneous observations into a single risk score.

Cross-View Joint Reasoning. After all three specialized agents complete their investigations, the Judge Agent receives their final structured reports and jointly considers supporting evidence, potential discrepancies in interpretation, and missing information across views. It then produces a malware decision together with an inspectable rationale and an interpretable evidence chain. In this way, Moirae progressively transforms dispersed runtime observations

into behavior-level risk findings and ultimately an application-level decision, enabling us to investigate the generalization potential of evidence-based reasoning under distribution shifts.

4 Method Implementation

4.1 Controlled Dynamic Exploration and Runtime Evidence Collection

For an application under analysis $\mathcal{A}$, `Moirae` executes an automated exploration workflow in an Android emulator and collects runtime artifacts at multiple semantic levels. The system uses DroidBot [20] to record the interaction process, obtaining a set of screenshots $\mathcal{V} = \{v_i\}$, UI states $\mathcal{S} = \{s_i\}$, and interaction events $\mathcal{E} = \{e_k\}$. Each UI state contains information such as text associated with interface-hierarchy nodes, resource identifiers, component attributes, and the foreground Activity. An interaction event is represented as $e_k = \langle k, s_k^-, a_k, s_k^+, t_k^{\mathrm{UI}} \rangle$, where k denotes the event index, a_k records the performed action, s_k^- and s_k^+ denote the corresponding source and destination UI states, and t_k^{UI} is the recorded relative event time. State hashes and event indices establish retrievable relationships among screenshots, UI-state nodes, and interaction actions. Because automated exploration is constrained by execution time and behavior-triggering conditions, the resulting sequence represents only the interaction paths actually observed during execution.

The system simultaneously collects ART method traces $\mathcal{T} = \{c_j\}$, from which it extracts monitored method calls and their summaries while recording the trace collection status. Both dynamic exploration and runtime monitoring have inherent coverage limitations; therefore, subsequent analysis relies only on evidence that is actually captured. When ART traces are empty or unavailable, other successfully collected observations are retained for analysis.

4.2 Event-Conditioned Candidate Association and Hierarchical Evidence Retrieval

Observations from different semantic levels differ substantially in granularity. To avoid directly concatenating large volumes of raw runtime traces into the LLM input, `Moirae` constructs tool-accessible evidence indices based on state hashes, event indices, and API-call summaries. At the UI level, each recorded interaction is organized as $s_k^- \rightarrow e_k \rightarrow s_k^+$, allowing investigation agents to inspect behaviors such as button actions, transitions to permission dialogs, or unexpected foreground changes. At the API level, event timestamps are used to construct candidate retrieval windows over low-level runtime traces.

Specifically, preprocessing defines an event-conditioned forward window of duration Δt: $W_k = [t_k^{\mathrm{UI}}, t_k^{\mathrm{UI}} + \Delta t]$, $\widetilde{\mathcal{T}_k} = \{c_j \in \mathcal{T}_\Omega : t_j^{\mathrm{ART}} \in W_k\}$, where $\mathcal{T}_\Omega$ denotes the method-call records filtered according to a predefined set of system APIs of interest, Ω, and t_j^{ART} denotes the recorded relative time of call c_j. For each candidate window, the system stores the corresponding event index, temporal range, trace status, and available API-call summaries. When valid traces are available, detailed calls and their associated statistics can also be retrieved. Currently, UI and ART timestamps are normalized independently using the first DroidBot state and the first valid ART method record as their respective temporal origins.

To preserve fine-grained evidence while controlling the amount of information provided to the model at each step, `Moirae` provides hierarchical and paginated retrieval tools. `list_screens` and `inspect_screen` support browsing and inspecting screenshots. `list_ui_events`, `read_ui_state`, and `read_related_events` retrieve UI nodes and their associated state transitions. `list_events` and `read_api_calls` retrieve candidate API windows and call summaries by event index and pagination offset. When the summaries are insufficient, an agent may further access raw event information within its authorized scope. Retrieval results preserve provenance information such as `screen_index`, `event_id`, state hashes, and trace-status indicators, providing explicit observational grounding for subsequent risk interpretation.

4.3 ReAct-Based Specialized Behavioral Investigation

Different evidence views support different investigation objectives. Screenshots primarily reflect content presented to the user, UI states and events describe observable interaction processes, and ART method traces provide low-level execution signals. Given their differences in semantic granularity, sequence length, and missing-data patterns, `Moirae` employs three tool-augmented ReAct agents that operate in independent contexts. Each agent invokes dedicated tools over the preprocessed artifacts and investigates the evidence available to its assigned view.

Visual Evidence Investigation (Vision Agent). A_{vis} uses screenshot indexing and image-inspection tools to analyze visible application functionality, impersonation cues, deceptive interaction prompts, and benign-looking user-facing content. Each risk finding is grounded in screenshot indices and concrete visual observations.

Interaction Behavior Investigation (UI/Event Agent). A_{ui} uses UI hierarchies, event lists, and state-relation retrieval tools to examine state transitions in $s_k^- \rightarrow e_k \rightarrow s_k^+$, with particular attention to permission requests, unexpected transitions, and potentially deceptive interactions. By combining interface elements with recorded actions, the agent interprets interaction behaviors that were actually observed during execution.

Low-Level Execution Investigation (API Agent). A_{api} retrieves ART-call summaries from candidate event windows and can progressively expand available records through paginated access. It investigates execution signals related to sensitive-data access, network communication, SMS operations, dynamic loading, and other security-relevant system activities. The agent uses event indices and limited interaction metadata only to localize candidate windows; its risk assessment is grounded in the API summaries and trace-status information actually observed.

The three specialists independently produce structured investigation reports R_{vis}, R_{ui}, and R_{api}. Each report includes the APK identifier, reviewed objects, a set of behavioral findings, an overall risk level, and a textual summary. Each finding contains fields such as a risk label (`label`), severity (`severity`), supporting observations (`evidence`), and risk interpretation (`interpretation`). The evidence field explicitly references screenshot, event, or API-window identifiers.

4.4 Cross-View Joint Adjudication and Result Interpretation

After the three specialists complete their APK-level investigations, Moirae forwards their final reports to the Judge Agent for centralized decision making: $(\hat{y}, \hat{c}, q, \rho, C) = \mathcal{M}_{\text{judge}}(R_{vis}, R_{ui}, R_{api})$, where $\hat{y} \in \{\text{benign}, \text{malware}\}$ denotes the predicted binary label, $\hat{c}$ denotes the threat category, $q \in \{0, \ldots, 100\}$ is an LLM-generated confidence score, ρ denotes the decision rationale, and C represents a natural-language explanatory evidence chain. The Judge bases its decision on the observations and interpretations reported by the three specialists. It prioritizes risk findings with concrete supporting evidence while jointly considering corroborating signals and alternative explanations across different views.

5 Experimental Evaluation

5.1 Experimental Setup

We implement Moirae in Python 3.11 and run it on a host equipped with an Intel Core i7-12700 CPU and 32 GB RAM. The agents use MiniMax-M3 by default, while we further evaluate different Judge model configurations [2]. In addition, we include Qwen3.6-27b, deployed on an NVIDIA A100 (80 GB) GPU, as an additional comparison model. None of the LLMs undergo domain-specific fine-tuning, and all use their default inference settings. Dynamic analysis is performed using Android Emulator 36.2.12.0 and ADB 36.0.0, with a Pixel 3 XL AVD running an Android 9 (API 28) Google APIs x86 image. The emulator is configured with four CPU cores, 2 GB RAM, 6 GB storage, and a resolution of 1440 × 2960. Each APK is automatically explored by DroidBot for two minutes, and we use a 2-second event-conditioned forward window ($\Delta = 2$) for candidate API retrieval; its rationale is discussed in Appendix A.1.

5.1.1 Datasets. We evaluate Moirae on three representative datasets: AndroZoo [4], CICMalDroid2020 [22], and CIC-AndMal2017 [18]. For AndroZoo, we label applications using VirusTotal detections (≥10 for malware and 0 for benign). We randomly sample 15,216 applications from 2011–2016 for training (7,523 malware and 7,693 benign) and 630 dynamically executable applications from 2017–2021 for testing. CICMalDroid2020 and CIC-AndMal2017 are strictly reserved as unseen test datasets, with 312 and 375 APKs, respectively, successfully executed during dynamic analysis, to evaluate cross-domain generalization. Usable sample counts vary across baselines due to different preprocessing requirements. Details on dynamic artifact availability and sample coverage are provided in Appendix A.2.

5.1.2 Baseline Methods. We select four categories of representative Android malware detection methods as baselines. 1) **Static Structural and Behavioral Models:** MaMaDroid [25] models application behavior through API-call transitions. MalScan [31] performs detection using centrality features derived from function call graphs. 2) **Adaptive and Drift-Resilient Models:** DroidEvolver [33] performs online adaptation using pseudo-labels. CL-Malware [8] combines contrastive learning with active learning to mitigate concept drift. 3) **Dynamic and Hybrid Analysis Models:** DroidCat [7] detects malware using runtime method calls and inter-component communication features. AndroPyTool [23] combines static and dynamic features with ensemble learning for classification. 4) **LLM-Assisted Detection Models:** AppPoet [43] uses LLMs to extract semantic information from multi-view static features and performs final detection through neural-network-based feature fusion.

[2] GLM-5.1, Gemini, and MiniMax-M3 were accessed via the APIs provided by SiliconFlow (https://siliconflow.cn/), OpenRouter (https://openrouter.ai/), and MiniMax (https://www.minimax.io/), respectively.

5.2 Research Questions (RQs)

To evaluate the effectiveness and applicability of Moirae, we investigate the following four research questions:

- **RQ1 (Effectiveness):** How does Moirae compare with existing Android malware detection approaches under different data distributions?
- **RQ2 (Temporal Generalization):** As Android malware continues to evolve, can Moirae maintain stable detection performance across different time periods?
- **RQ3 (Evidence and Ablation Analysis):** How do different dynamic evidence views and LLM model choices affect the detection performance of Moirae?
- **RQ4 (Efficiency and Reliability):** What are the token consumption, inference latency, and operational reliability of Moirae?

5.3 RQ1: Effectiveness

To evaluate the overall detection effectiveness of Moirae, we compare it with static structural analysis, adaptive learning, dynamic analysis, and LLM-assisted detection methods across three test datasets from different sources. Table 1 reports the results. Moirae achieves competitive performance across all three test sets. Specifically, Moirae (G) achieves F1 scores of 87.71%, 92.15%, and 90.64% on AndroZoo, CICMalDroid2020, and CIC-AndMal2017, respectively. On the two CIC datasets, the strongest source-domain-trained baseline is CL-Malware, which achieves F1 scores of 88.58% and 73.77%, respectively. In comparison, Moirae (G) improves F1 by 3.57 and 16.87 percentage points, demonstrating stronger detection performance across different data distributions.

Different categories of baselines exhibit substantial performance variation in cross-dataset evaluation. For example, CL-Malware, AppPoet, and DroidEvolver achieve F1 scores of 88.58%, 83.64%, and 76.65%, respectively, on CICMalDroid2020, but their performance decreases to 73.77%, 71.24%, and 65.89% on CIC-AndMal2017. Dynamic-analysis methods exhibit similar limitations: DroidCat achieves F1 scores of 76.51% and 61.42% on the two datasets. These results indicate that existing approaches can vary considerably in effectiveness across test distributions, whereas Moirae maintains consistently high F1 scores in both evaluations.

A closer examination of Precision and Recall further reveals differences in detection behavior. On CICMalDroid2020, CL-Malware achieves the highest Recall of 97.44%, but its Precision is only 81.20%. In comparison, Moirae (G) achieves a better balance, with 91.00% Precision and 93.33% Recall, resulting in a higher overall F1 score. On CIC-AndMal2017, Moirae (G) achieves 90.12% Precision and 91.18% Recall, both exceeding CL-Malware's 66.05% Precision and 83.53% Recall. This suggests that Moirae can maintain a high malware detection rate while better controlling false positives.

Table 1: Cross-dataset Android malware detection performance (%). The best result for each metric is shown in bold, and the second-best result is underlined. M denotes a Judge Agent using `MiniMax-M3`, while G denotes `GLM-5.1`.

Approach	CIC-AndMal2017				CICMalDroid2020				AndroZoo (2017–2021)			
	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1
MaMaDroid (Family)	44.53	42.15	60.00	49.51	31.09	43.75	35.90	39.44	70.00	68.83	82.14	71.14
MaMaDroid (Package)	51.47	47.52	67.65	55.83	60.26	64.49	81.03	71.82	76.22	72.14	83.27	75.14
MalScan (KNN-1)	52.53	48.52	77.06	59.55	60.90	63.88	86.15	73.36	70.20	66.41	69.33	66.58
MalScan (KNN-3)	50.40	47.08	75.88	58.11	58.01	62.31	83.08	71.21	73.66	68.83	74.09	70.30
MalScan (RF)	42.13	40.00	55.29	46.42	35.90	48.66	46.67	47.64	61.42	65.87	47.96	54.30
DroidEvolver	61.07	54.65	82.94	65.89	66.03	67.18	89.23	76.65	78.69	73.45	69.36	70.62
CL-Malware	73.07	66.05	83.53	73.77	84.29	81.20	**97.44**	88.58	81.74	81.88	84.34	79.93
DroidCat	65.20	62.12	60.74	61.42	72.76	83.82	70.37	76.51	73.77	72.49	70.78	69.47
AndroPyTool (StaticRF)	53.88	50.00	71.68	58.91	66.54	72.94	85.03	78.52	80.03	74.20	81.86	75.13
AndroPyTool (BaggingRF)	53.06	49.21	54.87	51.88	46.92	68.99	47.59	56.33	68.22	60.65	75.13	63.44
AndroPyTool (Fusion)	53.88	50.00	71.68	58.91	65.77	73.33	82.35	77.58	77.78	70.17	82.15	72.66
AppPoet	70.93	64.59	79.41	71.24	76.92	75.10	94.36	83.64	79.57	78.55	77.33	76.00
`Moirae` (M)	86.67	**90.54**	78.82	84.28	88.46	**92.51**	88.72	90.58	87.21	**94.69**	77.01	84.90
`Moirae` (G)	**91.47**	90.12	**91.18**	**90.64**	**90.06**	91.00	93.33	**92.15**	**89.10**	90.95	**84.77**	**87.71**

> Answer to RQ1: `Moirae` achieves strong overall detection performance across all three test datasets, with a particularly pronounced cross-dataset F1 advantage on CIC-AndMal2017. Compared with existing feature-learning, dynamic-analysis, and LLM-assisted approaches, `Moirae` provides a competitive solution for Android malware detection under different data distributions.

5.4 RQ2: Temporal Generalization

To evaluate the temporal generalization of `Moirae`, we conduct out-of-time testing using AndroZoo applications from 2017 to 2021 and compare the performance of different detection methods.

As shown in Figure 3, most baseline models exhibit a clear decline in detection performance as the test year advances. Although MaMaDroid and CL-Malware achieve strong performance in the earlier years, their F1 scores decrease from 94.89% and 91.94% in 2017 to 33.77% and 42.11% in 2021, corresponding to drops of 61.12 and 49.83 percentage points, respectively. Similarly, the F1 score of the LLM-assisted AppPoet decreases from 88.52% to 36.92%. These results indicate that, despite different feature representations and learning strategies, detectors constructed from historical data can still suffer substantial degradation when applied to applications from later years.

In contrast, `Moirae` maintains relatively stable detection performance across the five-year period. Its F1 score remains between 84.55% and 90.08%. In 2021, it achieves 94.38% Accuracy and 86.49% F1, exceeding the highest reported baseline F1 of 65.31% achieved by DroidCat. Further analysis shows that baseline degradation is reflected not only in missed malware samples but also in increased false positives. For example, in 2021, CL-Malware and `Moirae` both achieve a Recall of 84.21%, whereas their Precision values are 28.07% and 88.89%, respectively. While the two methods detect a similar proportion of malware samples, `Moirae` distinguishes benign applications more effectively.

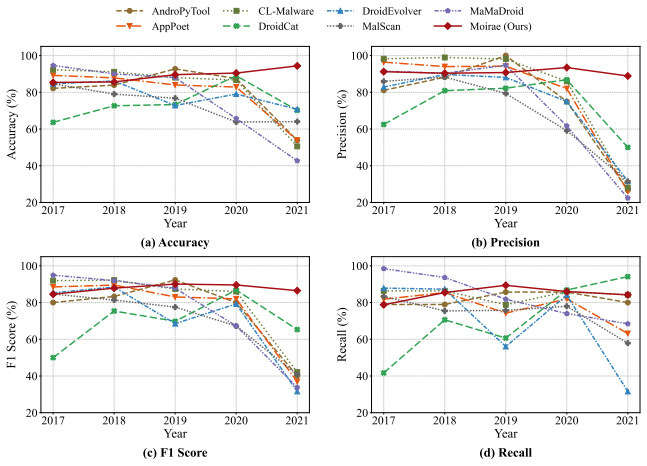


Figure 3: Temporal generalization performance on AndroZoo applications from 2017 to 2021.

> Answer to RQ2: Most baselines exhibit substantial performance degradation when evaluated on Android applications from later years, whereas `Moirae` maintains relatively stable F1 and Precision over the five-year period. These results demonstrate stronger temporal generalization under continuously evolving Android malware distributions.

5.5 RQ3: Evidence and Ablation Analysis

We evaluate the complementarity of dynamic evidence and the effectiveness of joint reasoning through cross-agent evidence analysis, modality ablation, and Judge model comparison.

Cross-Agent Evidence Complementarity. Table 2 summarizes the distribution of specialist risk support among malware samples and the corresponding Judge detection rates. Different specialists exhibit substantial variation in the risk signals they identify for the same malware sample. On CIC-AndMal2017, CICMalDroid2020, and AndroZoo, 63.53%, 54.36%, and 76.20% of malware samples, respectively, receive medium- or high-risk support from only one or two specialists. This observation suggests that no individual evidence view covers all suspicious behaviors. Among the three specialists, the UI/Event Agent consistently provides the highest support rate, ranging from 72.67% to 89.74%, whereas the API Agent provides support for only 27.65% of malware samples on AndroZoo. These differences reflect the complementary coverage of risk signals across dynamic evidence views.

The Judge results further show that malware detection rates generally increase with the number of specialists reporting suspicious evidence. When only one specialist reports medium or high risk, the Judge detects 67.50%, 79.07%, and 91.67% of malware samples on the three datasets, respectively. When at least two specialists provide supporting evidence, the detection rate exceeds 96% on all datasets. This observation indicates a strong association between cross-view evidence agreement and the final malware decision. At the same time, the Judge can still identify some malware samples even when none of the specialists assigns a medium or high risk level, suggesting that final adjudication is not equivalent to simple voting over specialist risk levels.

Table 2: Cross-agent risk support and Judge evidence accumulation for malware samples. Suspicious agents report medium or high risk.

(a) Distribution of suspicious agents among all malware samples			
Evidence-Support Pattern	**CIC-AndMal2017**	**CICMalDroid2020**	**AndroZoo**
No agent support	13.53%	6.15%	14.79%
One agent only	25.88%	25.13%	46.30%
Two agents only	37.65%	29.23%	29.90%
All three agents	22.94%	39.49%	9.00%
Vision Agent support	40.00%	64.10%	32.80%
UI/Event Agent support	75.29%	89.74%	72.67%
API Agent support	54.71%	48.21%	27.65%
(b) Judge evidence accumulation in successfully adjudicated malware			
Number of Suspicious Agents	**CIC-AndMal2017**	**CICMalDroid2020**	**AndroZoo**
0	14.29%	11.11%	26.09%
1	67.50%	79.07%	91.67%
2	96.72%	100.00%	98.92%
3	100.00%	100.00%	100.00%

Dynamic Evidence Module Ablation. To further quantify the contribution of each evidence view, we fix `MiniMax-M3` on CICMalDroid2020 and either remove or retain individual Vision, UI/Event, and API evidence modules. The results are reported in Table 3. The full model achieves an F1 score of 90.58%, outperforming all ablated configurations. Removing Vision, UI/Event, and API evidence decreases F1 by 1.87, 9.88, and 2.05 percentage points, respectively.

Table 3: Evidence-module ablation results on CICMalDroid2020 using `MiniMax-M3`. A ✓ indicates that the corresponding evidence module is retained.

Evidence Modules			**Performance (%)**			
Vision	**UI/Event**	**API**	**Acc**	**Pre**	**Rec**	**F1**
✓	✓	✓	**88.46**	92.51	**88.72**	**90.58**
	✓	✓	86.22	90.86	86.67	88.71
✓		✓	78.85	93.88	70.77	80.70
✓	✓		86.22	92.22	85.13	88.53
✓			78.53	95.07	69.23	80.12
	✓		86.54	90.05	88.21	89.12
		✓	70.19	**95.54**	54.87	69.71

These results indicate that all three evidence views contribute to final detection performance, with UI/Event evidence having the largest individual impact.

The single-view results provide further insight. UI/Event evidence alone achieves an F1 score of 89.12%, substantially higher than Vision (80.12%) and API (69.71%). However, although the API-only configuration achieves a Recall of only 54.87%, it obtains the highest Precision of 95.54%, suggesting that the suspicious calls it captures can provide highly discriminative evidence when available. In addition, some two-view combinations perform slightly worse than the UI/Event-only configuration, showing that adding additional evidence sources does not necessarily yield monotonic performance gains. Nevertheless, the full model still improves F1 by 1.46 percentage points over the strongest single-view configuration, demonstrating the complementary value of heterogeneous dynamic evidence in overall detection.

Representative Behavioral Cases. To further examine how multi-view dynamic evidence contributes to identifying concrete malicious behaviors, we select representative samples from the three datasets for case studies. These cases illustrate both situations in which evidence from different views corroborates one another and situations in which visible interface and interaction cues support the final decision despite incomplete API traces. The complete model outputs and evidence visualizations for the three cases are provided in Appendix A.3.

Judge Model Sensitivity. We further replace the LLMs used by the evidence specialists and the Judge to evaluate the sensitivity of `Moirae` to different model configurations. The results are shown in Table 4. When all three evidence specialists use `MiniMax-M3`, `GLM-5.1` achieves the highest F1 score of 92.15% as the Judge, improving by 1.57 percentage points over the homogeneous `MiniMax-M3` configuration. In contrast, the `MiniMax-M3` Judge achieves higher Precision (92.51%), whereas the `GLM-5.1` Judge achieves higher Recall (93.33%), indicating a Precision–Recall trade-off across adjudication models.

When the evidence specialists are replaced with `Qwen3.6-27b`, the `GLM-5.1` Judge still achieves the highest F1 within this group (89.15%), but the overall performance remains lower than that of the

corresponding configurations using `MiniMax-M3` specialists. Moreover, when `Qwen3.6-27b` is used for both evidence investigation and adjudication, Recall reaches 97.44%, while Precision decreases to 80.51%. These results indicate that detection effectiveness depends not only on the reasoning capability of the final Judge but also on the quality of upstream evidence reports and the specific combination of models.

Table 4: Model ablation results on CICMalDroid2020. Evidence specialists use either `MiniMax-M3` or locally deployed `Qwen3.6-27b`, while the Judge uses different LLMs.

Evidence Agents	Judge Agent	Acc	Pre	Rec	F1
MiniMax-M3	MiniMax-M3	88.46	**92.51**	88.72	**90.58**
	Gemini-3.1-Pro	85.26	92.09	83.59	87.63
	GLM-5.1	**90.06**	91.00	93.33	92.15
	GLM-4.7-Flash	87.18	91.44	87.69	89.53
Qwen3.6-27b	Qwen3.6-27b	83.65	80.51	**97.44**	88.17
	GLM-5.1	85.25	82.53	96.92	89.15
	GLM-4.7-Flash	78.21	75.30	96.92	84.75
	MiniMax-M3	84.94	84.58	92.82	88.51

> Answer to RQ3: Different dynamic evidence views provide complementary coverage of risk signals, with UI/Event evidence playing a key role in detection. The case studies further demonstrate the value of cross-view evidence association and joint reasoning under partial observability. In addition, different LLM configurations exhibit substantial performance variation, indicating that both evidence organization and joint adjudication are important factors affecting the effectiveness of `Moirae`.

5.6 RQ4: Efficiency and Reliability

To evaluate the practical overhead and reliability of `Moirae`, we analyze token consumption and end-to-end inference latency. Table 5 reports the average token consumption of each agent across the three datasets. The evidence specialists exhibit substantial differences in computational overhead. In particular, the API Agent consumes an average of 352.31k–515.91k tokens, accounting for 68.73%–71.73% of the total token consumption of the four agents and constituting the primary source of inference cost. This overhead mainly arises from the scale of low-level execution traces and the corresponding investigation process. Among the three evidence specialists, the Vision Agent incurs the lowest token cost, while the UI/Event Agent lies between the two, reflecting the asymmetric input scales and analysis requirements across evidence views.

The token distributions in Appendix A.4 reveal substantial cost variations across malware families. In CIC-AndMal2017, Adware (766k) and Scareware (742k) consume considerably more tokens than Benign (397k) and Ransomware (372k), while SMSmalware incurs the highest cost in CICMalDroid2020 (835k). These differences are consistent with the behavioral complexity of different malware categories. For example, Adware and SMS-based malware frequently trigger UI interactions and extensive API call sequences, resulting in longer behavioral traces. In contrast, Ransomware typically performs rapid file-encryption operations shortly after execution, producing shorter behavioral trajectories and requiring fewer evidence tokens.

Table 5: Average total token consumption.

Metric	CIC-AndMal2017	CICMalDroid2020	AndroZoo
Vision avg. total tokens	31.80k	39.47k	35.15k
UI/Event avg. total tokens	111.02k	148.82k	163.27k
API avg. total tokens	352.31k	515.91k	450.65k
Judge avg. total tokens	13.88k	15.00k	6.61k

Table 6: Comparison of time consumption.

Dataset	`Moirae` E2E	AppPoet Cumulative	AppPoet / `Moirae`
CIC-AndMal2017	**216.59** s	416.06 s	1.92×
CICMalDroid2020	**181.55** s	236.45 s	1.30×
AndroZoo	**222.18** s	363.56 s	1.64×

Table 6 reports the average processing time of `Moirae` and AppPoet. On CIC-AndMal2017, CICMalDroid2020, and AndroZoo, the end-to-end processing time of `Moirae` is 216.59 s, 181.55 s, and 222.18 s per sample, respectively, whereas the cumulative processing time of AppPoet is 416.06 s, 236.45 s, and 363.56 s. Under the current measurement settings, AppPoet requires 1.92×, 1.30×, and 1.64× the processing time of `Moirae` on the three datasets, respectively. These results indicate that `Moirae` maintains competitive processing efficiency under the evaluated implementation. A detailed breakdown of `Moirae`'s runtime is provided in Appendix A.4.

> Answer to RQ4: The primary token overhead of `Moirae` arises from API evidence analysis, while its division of evidence-investigation responsibilities allows the Judge to operate on comparatively compact specialist reports. Under the current experimental settings, `Moirae` requires less processing time than the cumulative runtime of AppPoet.

6 Conclusion

This paper presents `Moirae`, a multimodal multi-agent framework for Android malware detection. By organizing complementary dynamic evidence from visual content, UI/Event interactions, and API calls, `Moirae` combines specialized ReAct agents with a centralized Judge to perform malware detection grounded in behavioral context. Experimental results show that `Moirae` achieves an F1 score of 92.15% without target-domain fine-tuning and exhibits strong temporal generalization. Ablation studies and case analyses further demonstrate the complementary value of multi-view dynamic evidence. Nevertheless, `Moirae` remains limited by the incomplete coverage of dynamic exploration. Future work will investigate more effective behavior-triggering strategies to further improve detection capability in complex environments.

References

[1] Yousra Aafer, Wenliang Du, and Heng Yin. 2013. Droidapiminer: Mining api-level features for robust malware detection in android. In *International conference on security and privacy in communication systems*. Springer, 86–103.

[2] Vitor Afonso, Antonio Bianchi, Yanick Fratantonio, Adam Doupé, Mario Polino, Paulo De Geus, Christopher Kruegel, Giovanni Vigna, et al. 2016. Going native: Using a large-scale analysis of android apps to create a practical native-code sandboxing policy. In *The Network and Distributed System Security Symposium 2016*. 1–15.

[3] Faridah Akinotcho, Lili Wei, and Julia Rubin. 2025. Mobile application coverage: The 30% curse and ways forward. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2751–2763.

[4] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *Proceedings of the 13th International Conference on Mining Software Repositories* (Austin, Texas) *(MSR '16)*. ACM, New York, NY, USA, 468–471.

[5] Philipp Beer, Sebastian Roth, Martina Lindorfer, and Marco Squarcina. 2026. Plain Text, Plain Risks: Measuring {HTTP} Inclusion in Android {WebViews} at Scale. In *35th USENIX Security Symposium (USENIX Security 26)*. 2685–2704.

[6] Parnika Bhat and Kamlesh Dutta. 2019. A survey on various threats and current state of security in android platform. *ACM Computing Surveys (CSUR)* 52, 1 (2019), 1–35.

[7] Haipeng Cai, Na Meng, Barbara Ryder, and Daphne Yao. 2018. Droidcat: Effective android malware detection and categorization via app-level profiling. *IEEE Transactions on Information Forensics and Security* 14, 6 (2018), 1455–1470.

[8] Yizheng Chen, Zhoujie Ding, and David Wagner. 2023. Continuous learning for android malware detection. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1127–1144.

[9] Yoonjung Choi, Woonghee Lee, and Junbeom Hur. 2024. Phishinwebview: Analysis of anti-phishing entities in mobile apps with webview targeted phishing. In *Proceedings of the ACM Web Conference 2024*. 1923–1932.

[10] Lei Cui, Jiancong Cui, Yuede Ji, Zhiyu Hao, Lun Li, and Zhenquan Ding. 2023. Api2vec: Learning representations of api sequences for malware detection. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 261–273.

[11] Parvez Faruki, Ammar Bharmal, Vijay Laxmi, Vijay Ganmoor, Manoj Singh Gaur, Mauro Conti, and Muttukrishnan Rajarajan. 2014. Android security: a survey of issues, malware penetration, and defenses. *IEEE communications surveys & tutorials* 17, 2 (2014), 998–1022.

[12] Jiayin Feng, Limin Shen, Zhen Chen, Yu Lei, and Hui Li. 2025. HGDetector: A hybrid Android malware detection method using network traffic and Function call graph. *Alexandria Engineering Journal* 114 (2025), 30–45.

[13] Alessandra Gorla, Ilaria Tavecchia, Florian Gross, and Andreas Zeller. 2014. Checking app behavior against app descriptions. In *Proceedings of the 36th international conference on software engineering*. 1025–1035.

[14] Yiling He, Hongyu She, Xingzhi Qian, Xinran Zheng, Zhuo Chen, Zhan Qin, and Lorenzo Cavallaro. 2025. On benchmarking code llms for android malware analysis. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 153–160.

[15] Jianjun Huang, Xiangyu Zhang, Lin Tan, Peng Wang, and Bin Liang. 2014. Asdroid: Detecting stealthy behaviors in android applications by user interface and program behavior contradiction. In *Proceedings of the 36th international conference on software engineering*. 1036–1046.

[16] Lu Huang, Jingfeng Xue, Yong Wang, Junbao Chen, and Tianwei Lei. 2024. Strengthening LLM ecosystem security: Preventing mobile malware from manipulating LLM-based applications. *Information Sciences* 681 (2024), 120923.

[17] TaeGuen Kim, BooJoong Kang, Mina Rho, Sakir Sezer, and Eul Gyu Im. 2018. A multimodal deep learning method for android malware detection using various features. *IEEE Transactions on Information Forensics and Security* 14, 3 (2018), 773–788.

[18] Arash Habibi Lashkari, Andi Fitriah A Kadir, Laya Taheri, and Ali A Ghorbani. 2018. Toward developing a systematic approach to generate benchmark android malware datasets and classification. In *2018 International Carnahan conference on security technology (ICCST)*. ieee, 1–7.

[19] Jin Li, Lichao Sun, Qiben Yan, Zhiqiang Li, Witawas Srisa-An, and Heng Ye. 2018. Significant permission identification for machine-learning-based android malware detection. *IEEE Transactions on Industrial Informatics* 14, 7 (2018), 3216–3225.

[20] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2017. Droidbot: a lightweight ui-guided test input generator for android. In *2017 IEEE/ACM 39th international conference on software engineering companion (ICSE-C)*. IEEE, 23–26.

[21] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2024. Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[22] Samaneh Mahdavifar, Andi Fitriah Abdul Kadir, Rasool Fatemi, Dima Alhadidi, and Ali A Ghorbani. 2020. Dynamic android malware category classification using semi-supervised deep learning. In *2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*. IEEE, 515–522.

[23] Alejandro Martín, Raúl Lara-Cabrera, and David Camacho. 2019. Android malware detection through hybrid features fusion and ensemble classifiers: The AndroPyTool framework and the OmniDroid dataset. *Information Fusion* 52 (2019), 128–142.

[24] Zhaoyi Meng, Jiale Zhang, Jiaqi Guo, Wansen Wang, Wenchao Huang, Jie Cui, Hong Zhong, and Yan Xiong. 2025. Detecting android malware by visualizing app behaviors from multiple complementary views. *IEEE Transactions on Information Forensics and Security* 20 (2025), 2915–2929.

[25] Lucky Onwuzurike, Enrico Mariconti, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2019. Mamadroid: Detecting android malware by building markov chains of behavioral models (extended version). *ACM Transactions on Privacy and Security (TOPS)* 22, 2 (2019), 1–34.

[26] Xingzhi Qian, Xinran Zheng, Yiling He, Shuo Yang, and Lorenzo Cavallaro. 2025. Lamd: Context-driven android malware detection and classification with llms. In *2025 IEEE Security and Privacy Workshops (SPW)*. IEEE, 126–136.

[27] Junyang Qiu, Qing-Long Han, Wei Luo, Lei Pan, Surya Nepal, Jun Zhang, and Yang Xiang. 2022. Cyber code intelligence for android malware detection. *IEEE Transactions on Cybernetics* 53, 1 (2022), 617–627.

[28] Kimberly Tam, Salahuddin J Khan, Aristide Fattori, and Lorenzo Cavallaro. 2015. CopperDroid: automatic reconstruction of Android malware behaviors. In *Ndss*, Vol. 15. 8–11.

[29] Junwei Tang, Sijie Zhou, Tao Peng, Xiaoyun Yan, Xinrong Hu, and Wenlong Tian. 2024. DTDroid: adversarial packed android malware detection based on traffic and dynamic behavioral. *IEEE Internet of Things Journal* 12, 3 (2024), 2646–2658.

[30] Lihong Tang, Xiao Chen, Sheng Wen, Li Li, Marthie Grobler, and Yang Xiang. 2023. Demystifying the evolution of android malware variants. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2023), 3324–3341.

[31] Yueming Wu, Xiaodi Li, Deqing Zou, Wei Yang, Xin Zhang, and Hai Jin. 2019. Malscan: Fast market-wide mobile malware scanning by social-network centrality analysis. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 139–150.

[32] Jiayun Xu, Yingjiu Li, Robert H Deng, and Ke Xu. 2020. Sdac: A slow-aging solution for android malware detection using semantic distance based api clustering. *IEEE transactions on dependable and secure computing* 19, 2 (2020), 1149–1163.

[33] Ke Xu, Yingjiu Li, Robert Deng, Kai Chen, and Jiayun Xu. 2019. Droidevolver: Self-evolving android malware detection system. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 47–62.

[34] Pei Yan, Shunquan Tan, Miaohui Wang, and Jiwu Huang. 2025. Prompt engineering-assisted malware dynamic analysis using gpt-4. *IEEE Transactions on Dependable and Secure Computing* (2025).

[35] Hongyu Yang, Youwei Wang, Liang Zhang, Xiang Cheng, and Ze Hu. 2024. A novel Android malware detection method with API semantics extraction. *Computers & Security* 137 (2024), 103651.

[36] Jiyun Yang, Hanwei Li, Lijun He, Tao Xiang, and Yujie Jin. 2024. MDADroid: A novel malware detection method by constructing functionality-API mapping. *Computers & Security* 146 (2024), 104061.

[37] Limin Yang, Wenbo Guo, Qingying Hao, Arridhana Ciptadi, Ali Ahmadzadeh, Xinyu Xing, and Gang Wang. 2021. {CADE}: Detecting and explaining concept drift samples for security applications. In *30th USENIX Security Symposium (USENIX Security 21)*. 2327–2344.

[38] Wei Yang, Xusheng Xiao, Benjamin Andow, Sihan Li, Tao Xie, and William Enck. 2015. Appcontext: Differentiating malicious and benign mobile app behaviors using context. In *2015 IEEE/ACM 37th IEEE international conference on software engineering*, Vol. 1. IEEE, 303–313.

[39] Suleiman Y Yerima, Mohammed K Alzaylaee, and Sakir Sezer. 2019. Machine learning-based dynamic analysis of Android apps with improved code coverage. *EURASIP Journal on Information Security* 2019, 1 (2019), 4.

[40] Xueying Zeng, Youquan Xian, Sihao Liu, Xudong Mou, Yanze Li, Lei Cui, and Bo Li. 2026. MARD: A Multi-Agent Framework for Robust Android Malware Detection. *arXiv preprint arXiv:2604.25264* (2026).

[41] Guangyu Zhang, Xixuan Wang, Shiyu Sun, Peiyan Xiao, Kun Sun, and Yanhai Xiong. 2025. TraceRAG: A LLM-Based Framework for Explainable Android Malware Detection and Behavior Analysis. *arXiv preprint arXiv:2509.08865* (2025).

[42] Sanfeng Zhang, Heng Su, Hongxian Liu, and Wang Yang. 2025. MPDroid: A multimodal pre-training Android malware detection method with static and dynamic features. *Computers & Security* 150 (2025), 104262.

[43] Wenxiang Zhao, Juntao Wu, and Zhaoyi Meng. 2025. Apppoet: Large language model based android malware detection via multi-view prompt engineering. *Expert Systems with Applications* 262 (2025), 125546.

[44] Ce Zhou, Yilun Liu, Weibin Meng, Shimin Tao, Weinan Tian, Feiyu Yao, Xiaochun Li, Tao Han, Boxing Chen, and Hao Yang. 2025. Srdc: Semantics-based

ransomware detection and classification with llm-assisted pre-training. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 28566–28574.

A Appendix

A.1 Rationale for the Event-Window Length

To evaluate the suitability of the event-window length, we further analyze the temporal intervals between adjacent UI events and the coverage of API evidence within the corresponding windows. The results are reported in Table 8.

First, across all three datasets, every valid interval between adjacent UI events is at least 2 seconds. This indicates that a 2-second window avoids internal overlap between neighboring event windows. Only 0.41%, 0.02%, and 0.34% of adjacent windows touch at their boundaries on CIC-AndMal2017, CICMalDroid2020, and AndroZoo, respectively. These results suggest that the selected window length preserves a short post-event observation range while reducing temporal ambiguity between different interaction events.

Second, although only 9.18%–12.55% of event windows contain at least one monitored API call, 68.00%–89.42% of APKs contain at least one event window with an API hit. This indicates that short event windows can provide candidate low-level behavioral cues for a subset of observed interactions without directly associating the complete API trace with every UI event.

It is important to note that the UI and ART traces currently do not share a common synchronization reference. Although 93.02% and 91.04% of valid ART samples in CIC-AndMal2017 and CICMalDroid2020, respectively, satisfy the overall duration-consistency check, this proportion is only 1.82% on AndroZoo. Therefore, event-window matches are used only for candidate API retrieval rather than as deterministic cross-layer associations.

A.2 Dynamic Evidence Collection and Event-Window Diagnostics

Table 7 summarizes the runtime artifacts collected from the three test datasets and their availability. In total, the system collects 10,095 valid screenshots, 9,066 non-empty UI states, and 26,140 valid interaction events. In contrast, the proportions of APKs with non-empty and parsable ART traces are 68.80%, 89.42%, and 78.57% for CIC-AndMal2017, CICMalDroid2020, and AndroZoo, respectively, indicating variation in the availability of low-level execution evidence across datasets. In addition, AndroPyTool and DroidCat are applicable to only subsets of the test samples. Differences between their valid evaluation subsets and the complete test sets may therefore also affect performance comparisons.

A.3 Behavioral Case Studies

To further illustrate the behavioral analysis process of `Moirae`, Table 9 summarizes the specialist risk assessments, Judge decisions, and key evidence for three representative malware samples. Figures 4–6 present concrete behavioral cues from lock-screen ransomware, a Banking Trojan, and adware, respectively, including interface impersonation, deceptive interactions, sensitive API calls, and abnormal page redirections. These cases illustrate how `Moirae` integrates dynamic evidence from different views to interpret potentially malicious behaviors and leverages other available observations when low-level execution evidence is limited.



Figure 4: Lock-screen ransomware case: cross-view evidence linking video-player impersonation, device-administrator authorization, and the ransom interface.

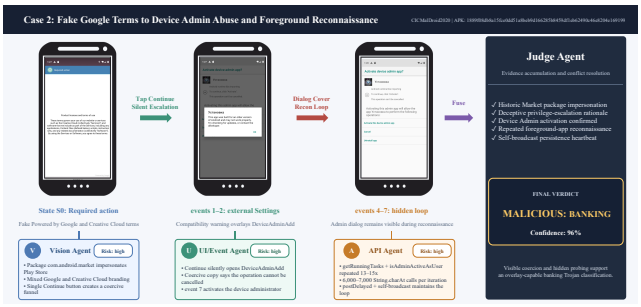


Figure 5: Banking Trojan case: a spoofed terms page, deceptive device-administrator authorization, and foreground-application reconnaissance.



Figure 6: Adware case: functional impersonation, a forced offer wall, and joint reasoning under incomplete API observations.

A.4 Token Usage and Execution Efficiency

To further examine the motivation and execution efficiency of the multi-agent design, we analyze token consumption across evidence views and the latency benefits of parallel investigation. Table 5 reports the average token usage of the four agents, while Figures 7–9 illustrate family-level variations, agent-wise cost composition, and cumulative input–output token distributions.

Evidence Workload Asymmetry. Token consumption varies considerably across malware families and evidence views, as shown in

Table 7: Statistics of dynamic execution and artifact availability across the test datasets. The last two columns report the numbers of valid evaluation samples for the corresponding baselines.

Dataset/Year	APK (Ben./Mal.)	Valid Screens	Non-Empty UI States	Valid UI Events	Non-Empty ART	AndroPyTool Valid Samples	DroidCat Valid Samples
AndroZoo	630 (319/311)	4,870	4,338	12,871	495 (78.57%)	339	550
CICMalDroid2020	312 (117/195)	2,599	2,430	6,826	279 (89.42%)	260	257
CIC-AndMal2017	375 (205/170)	2,626	2,298	6,443	258 (68.80%)	245	296

Table 8: Temporal consistency diagnostics between UI events and ART method calls. An API hit indicates that at least one monitored method-entry record falls within the corresponding 2-second event window.

Temporal Diagnostic Metric	CIC-AndMal2017	CICMalDroid2020	AndroZoo
Adjacent-event interval ≥ window length	6,068/6,068 (100.00%)	6,514/6,514 (100.00%)	12,241/12,241 (100.00%)
Adjacent windows touching only at closed endpoints	25/6,068 (0.41%)	1/6,514 (0.02%)	42/12,241 (0.34%)
Consistent UI/ART total duration (5 s tolerance)	240/258 (93.02%)	254/279 (91.04%)	9/495 (1.82%)
Events with at least one monitored API hit	776/6,443 (12.04%)	857/6,826 (12.55%)	1,181/12,871 (9.18%)
APKs with at least one event containing an API hit	255/375 (68.00%)	279/312 (89.42%)	494/630 (78.41%)

Table 9: Representative cross-view malicious-behavior cases from the three datasets. V, U, and A denote the Vision, UI/Event, and API Agents, respectively; H, M, and L denote high, medium, and low risk.

Dataset, and Type	Agent Risk	Judge Decision	Key Evidence and Decision Rationale
CIC-AndMal2017 Lock-Screen Ransomware	V=H U=H A=H	Ransomware 95%	The application disguises itself as a "Video Player" and immediately requests device-administrator privileges after launch. After authorization at `event_id=5`, the BACK action fails to exit the application, followed by a fake FBI page claiming device seizure, criminal charges, and monetary penalties. The API trace additionally records repeated `DevicePolicyManager.isAdminActive` checks. The Judge links application impersonation, administrator privilege escalation, exit prevention, and the ransom interface into a coherent lock-screen ransomware behavior chain.
CICMalDroid2020 Administrator Abuse and Foreground Reconnaissance	V=H U=H A=H	Banking 96%	The sample uses `com.android.market` to impersonate Android Market and presents a "Powered by Google" terms page that induces the user to click Continue. It then silently transitions to `DeviceAdminAdd` and completes device-administrator authorization. The API Agent simultaneously observes repeated `getRunningTasks` and `isAdminActiveAsUser` calls, together with delayed self-broadcasts. Based on brand impersonation, deceptive privilege escalation, and foreground-application reconnaissance, the Judge classifies the sample as a Banking Trojan with potential overlay-attack capabilities.
AndroZoo 2019 Deceptive Adware	V=H U=M A=L	Adware 92%	The application claims to be the "Cut Cut" photo editor but never provides the advertised editing functionality. Instead, it requires the user to install third-party applications or complete surveys before proceeding. The UI/Event Agent further observes that the offer wall redirects to an unrelated Baidu page without a corresponding user action, eventually exposing a generic Website-to-APK wrapper. The API Agent observes only routine WebView calls, while subsequent events suffer from coverage gaps. The Judge therefore classifies the sample as Adware based on functional impersonation, the forced offer wall, and silent third-party redirection, without treating the low-information API observations as benign counter-evidence.

Figure 7 and Figure 8. In CIC-AndMal2017, Adware, Scareware, and SMSmalware consume approximately 766k, 742k, and 646k tokens on average, respectively, compared with 397k for Benign and 372k for Ransomware. Similar variations appear in CICMalDroid2020, where Banking and SMSmalware consume approximately 782k and 835k tokens. These differences may arise from variations in observed UI interactions, API-call volumes, and tool-assisted investigation requirements.

More importantly, the API Agent accounts for 68.73%–71.73% of total token consumption across datasets, substantially exceeding the other evidence agents. This imbalance highlights the difficulty of handling heterogeneous evidence streams with markedly different analysis workloads. `Moirae` addresses this challenge by assigning each evidence view to a specialized agent with an independent investigation context, rather than combining all raw observations into a shared reasoning context.

Specialized Investigation and Joint Adjudication. As shown in Figure 9, cumulative input tokens generally reach 10^5–10^6, whereas output tokens are concentrated around 10^4. Instead of directly processing all raw evidence, the Judge integrates structured findings produced by the specialized agents. Consequently, the Judge accounts for only 1.01%–2.73% of the total token consumption. This separation allows evidence-intensive investigation and cross-view

adjudication to operate at different levels of information granularity, providing a practical motivation for the multi-agent architecture.

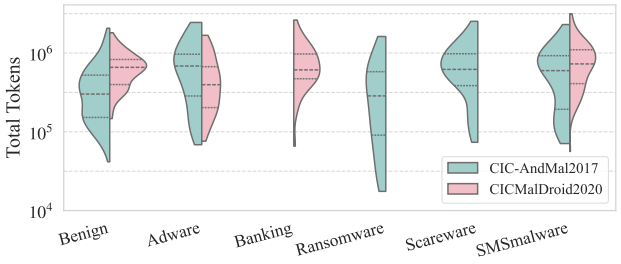


Figure 7: Distribution of total token consumption across datasets and malware families.

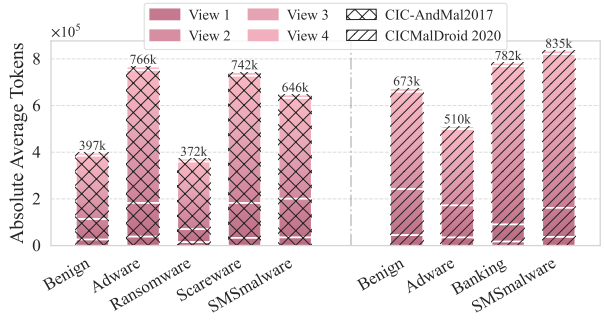


Figure 8: Average token consumption across malware families and its composition among the Vision, UI/Event, API, and Judge Agents.

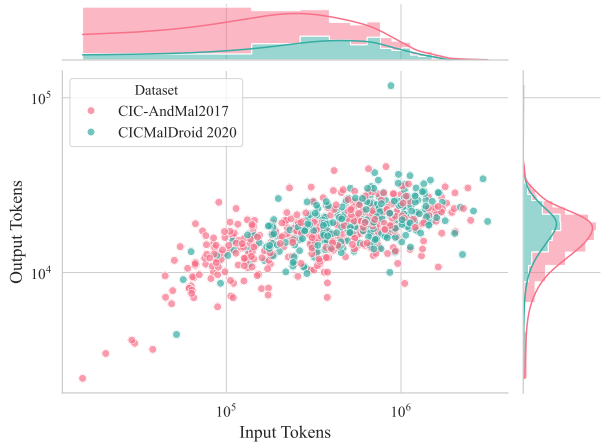


Figure 9: Relationship between cumulative input and output tokens across datasets on logarithmic axes.

Table 10: Cross-dataset system efficiency and inference reliability.

Metric	CIC-AndMal2017	CICMalDroid2020	AndroZoo
Avg. parallel evidence stage max(V, U, A)	159.63 s	150.27 s	191.29 s
Avg. Judge stage	56.84 s	31.13 s	27.96 s
Avg. orchestration overhead	0.12 s	0.16 s	2.93 s
Avg. end-to-end wall-clock time	216.59 s	181.55 s	222.18 s
Avg. serial-equivalent cumulative agent time	342.06 s	313.91 s	379.97 s
Serial-equivalent time / end-to-end wall-clock time	1.58×	1.73×	1.71×

Parallel Execution Efficiency. Table 10 further evaluates the execution efficiency of this design. With three evidence agents running in parallel, the average end-to-end latency is 216.59 s, 181.55 s, and 222.18 s on CIC-AndMal2017, CICMalDroid2020, and AndroZoo, respectively. The corresponding serial-equivalent cumulative agent times are 342.06 s, 313.91 s, and 379.97 s, yielding time ratios of 1.58×, 1.73×, and 1.71×. These results show that parallel investigation reduces the critical execution path relative to serial execution of the same agent calls.